



**CHANDIGARH**  
**UNIVERSITY**  
Discover. Learn. Empower.

**Subject Name: Frontend Technologies**

**Subject Code: 25CAH-655**

**Mini Project Report**

**On**

**UrbanForge 5.0: Advanced Living City AI Simulator**

**Submitted by:**

Name: Nancy

Uid: 25MCA20253

Class: MCA (General)

Section: 25MCA-2-A

**Submitted to:**

Ms. Deepali Saini

Assistant Professor

## Table of Content

| S.No. | Content                | Page No. |
|-------|------------------------|----------|
| 1     | Acknowledgement        | 1        |
| 2     | Introduction           | 2        |
| 3     | Objective              | 3        |
| 4     | Technologies Used      | 4        |
| 5     | Requirements           | 4-5      |
| 6     | Source Code & Output   | 5-8      |
| 7     | Conclusion/Future Work | 9        |
| 8     | References             | 10       |

## **Acknowledgement**

I take this opportunity to express my sincere gratitude to everyone who has supported and guided me throughout the completion of this project titled "UrbanForge 5.0: Advanced Living City AI Simulator", a Python and JavaScript-based interactive city building simulation.

I am deeply thankful to Ms. Deepali Saini for his valuable guidance, encouragement, and constant support during the development of this project. His insightful suggestions, motivation, and expert advice have been instrumental in enhancing the quality and success of my work.

I would also like to extend my heartfelt thanks to Chandigarh University and the Department of Master of Computer Applications (MCA) for providing an excellent academic environment, essential resources, and continuous support that made this project possible.

My special thanks go to my classmates, friends, and family members for their unwavering encouragement, patience, and moral support throughout every stage of this project.

Lastly, I wish to acknowledge the valuable assistance of various online learning platforms, GitHub repositories, AI tools such as Copilot, ChatGPT, Gemini, and the official documentation of Python, JavaScript, and jQuery UI, which greatly contributed to my learning and successful implementation of this project.

## **Introduction**

The UrbanForge 5.0: Advanced Living City AI Simulator is a fully browser-based interactive city-building simulation application developed without any server-side infrastructure. Built as a single self-contained HTML file, it runs directly in any modern web browser without installation.

This project integrates HTML5, CSS3, Bootstrap, jQuery, and jQuery UI to implement a rich multi-panel simulation environment featuring real-time economy, time-of-day effects, seasonal changes, disaster simulations, a technology research tree, city policy management, population tracking, and 33 unique SVG-illustrated placeable items.

The application includes a live economy engine (budget, income, upkeep), an energy grid system, building upgrades up to Level 3, zoning tools, a live minimap, population density heatmap, 13 achievements, 8 research technologies, 7 city policies, 5 smart connection types, and a Web Audio API ambient music system.

The system features four times of day (Morning, Afternoon, Evening, Night) and four seasons (Summer, Autumn, Winter with animated snowfall, Spring), each changing sky gradients, sun/moon position, building window glow, and ambient music. The disaster simulation system includes Thunderstorm (lightning bolts), Fire (particle animations), Earthquake (canvas shake), and Flood events, each with unique visual effects and city impact mechanics.

## Objectives

The main objectives of the UrbanForge 5.0 are to provide a fully interactive browser-based city simulation with realistic living systems. The project aims to demonstrate front-end engineering capabilities through visual simulation, real-time data, and complex interactive systems all contained in a single HTML file.

### Specific Objectives:

- 1. To build a complete drag-and-drop city canvas** using jQuery UI draggable/droppable with snap-to-grid, item placement, resizing, rotation, context menus, and auto-connection of roads.
- 2. To implement a real-time economy engine** where every building has build cost, daily income, and upkeep cost. Budget ticks every 10 seconds (1 game day), with deficit warnings and budget-locked palette items.
- 3. To design a living environment system** including 4 times of day with dynamic sky gradients, 4 seasons (including animated snowfall in winter), 230-star canvas for nighttime, and animated clouds.
- 4. To simulate city disasters** — Thunderstorm (lightning bolt SVG animations, rain overlay), Fire (CSS particle effects on buildings), Earthquake (canvas shake animation with structural damage), and Flood (financial and greenery penalties).
- 5. To implement a Technology Research Tree** with 8 unlockable technologies that permanently modify gameplay (green energy output +30%, AI governance +20% happiness, smart grid -20% energy consumption, etc.).
- 6. To provide city policy management** with 7 toggleable policies each carrying real economic and social tradeoffs affecting income, happiness, upkeep, and population growth.
- 7. To track and display live city statistics** across 8 dashboard tabs — Items, Links, History (with canvas thumbnail snapshots), Log, Analytics, Research, Policies, and Achievements — all fully functional and populated with live data.

## Technologies Used

The **UrbanForge 5.0** is developed using core web technologies and open-source libraries. All dependencies are loaded via CDN, making the application fully portable.

Table 1 Technologies Used

| S.no | Technology       | Purpose                                                                       |
|------|------------------|-------------------------------------------------------------------------------|
| 1    | HTML5 + CSS3     | Application structure, animations, theming, season CSS variables              |
| 2    | JavaScript (ES6) | Core application logic, economy engine, disaster system, achievements         |
| 3    | jQuery 3.7       | DOM manipulation, event handling, dynamic content rendering                   |
| 4    | jQuery UI 1.13   | Drag-and-drop, droppable canvas, resizable items                              |
| 5    | Bootstrap 5.3    | Responsive layout and utility classes                                         |
| 6    | Bootstrap Icons  | UI icons throughout the interface                                             |
| 7    | SVG (Inline)     | All 33 hand-crafted item illustrations (buildings, people, animals, vehicles) |

## Requirements

### Hardware Requirements

Table 2 Hardware requirements

| Component | Minimum                     |
|-----------|-----------------------------|
| Processor | Intel Core i3 or equivalent |
| RAM       | 4 GB                        |
| Storage   | 100 MB (browser cache)      |
| Display   | 1366×768 or higher          |

## Software Requirements

Table 3 Software requirements

| Component       | Specification                                                           |
|-----------------|-------------------------------------------------------------------------|
| Browser         | Chrome 90+, Firefox 88+, Edge 90+, Safari 14+                           |
| Internet        | Required for initial CDN load (jQuery, Bootstrap, Bootstrap Icons)      |
| Languages       | HTML5, CSS3, JavaScript ES6                                             |
| Libraries (CDN) | jQuery 3.7.1, jQuery UI 1.13.2, Bootstrap 5.3.3, Bootstrap Icons 1.11.3 |
| IDE             | VS Code / Notepad++ / Any text editor                                   |

## Source Code & Output

### Key Code Snippets

#### 1. Economy Tick Engine:

```
function economyTick() {
  const netIncome = S.income - S.upkeep;
  S.stats.budget += netIncome;
  if (S.activePolicies.has('pop_growth')) S.stats.pop += 10;
  if (S.activePolicies.has('green_tax'))
    S.stats.green = Math.min(100, S.stats.green + 5);
  S.cityDay++;
  renderBudget(); renderStats();
  buildPalette(); // refresh budget-locked items
  if (S.stats.budget < 0)
    toast('⚠ Budget deficit! Reduce upkeep or add income.');
```

#### 2. Building Upgrade System:

```
function upgradeItem(id) {
```

```
const rec = S.items.find(i => i.id === id);
const def = ITEMS.find(d => d.type === rec.type) || {};
const curLvl = S.itemLevels[id] || 1;
if (curLvl >= def.maxLvl) { toast('Already at max level!'); return; }
const upgCost = def.cost * 1.5 * curLvl;
if (S.stats.budget < upgCost) { toast('Not enough budget!'); return; }
S.stats.budget -= Math.round(upgCost);
S.itemLevels[id] = curLvl + 1;
const bonus = def.upgradeBonus || {};
if (bonus.pop) S.stats.pop += bonus.pop;
if (bonus.income) S.income += bonus.income;
if (bonus.happy) S.stats.happy = Math.min(100, S.stats.happy + bonus.happy);
renderStats(); renderBudget();
}
```

### 3. Canvas Minimap Rendering:

```
function updMinimap() {
  const c = document.getElementById('minimapCanvas');
  c.width = 128; c.height = 82;
  const ctx = c.getContext('2d');
  ctx.fillStyle = 'rgba(0,5,20,.7)'; ctx.fillRect(0, 0, 128, 82);
  S.conns.forEach(c => {
    const fr = S.items.find(i => i.id === c.from);
    const to = S.items.find(i => i.id === c.to);
    if (!fr || !to) return;
    ctx.strokeStyle = 'rgba(0,229,255,.3)'; ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.moveTo((fr.x / 1400) * 128, (fr.y / 900) * 82);
    ctx.lineTo((to.x / 1400) * 128, (to.y / 900) * 82);
    ctx.stroke();
  });
  S.items.forEach(item => {
    const def = ITEMS.find(d => d.type === item.type) || {};
    const cx = Math.round((item.x / 1400) * 128);
    const cy = Math.round((item.y / 900) * 82);
    ctx.fillStyle = def.cat === 'buildings' ? '#4a90e0' :
      def.cat === 'nature' ? '#2ecc71' : '#ffa726';
    ctx.beginPath(); ctx.arc(cx, cy, 2.5, 0, Math.PI * 2); ctx.fill();});}
```



## Output:-

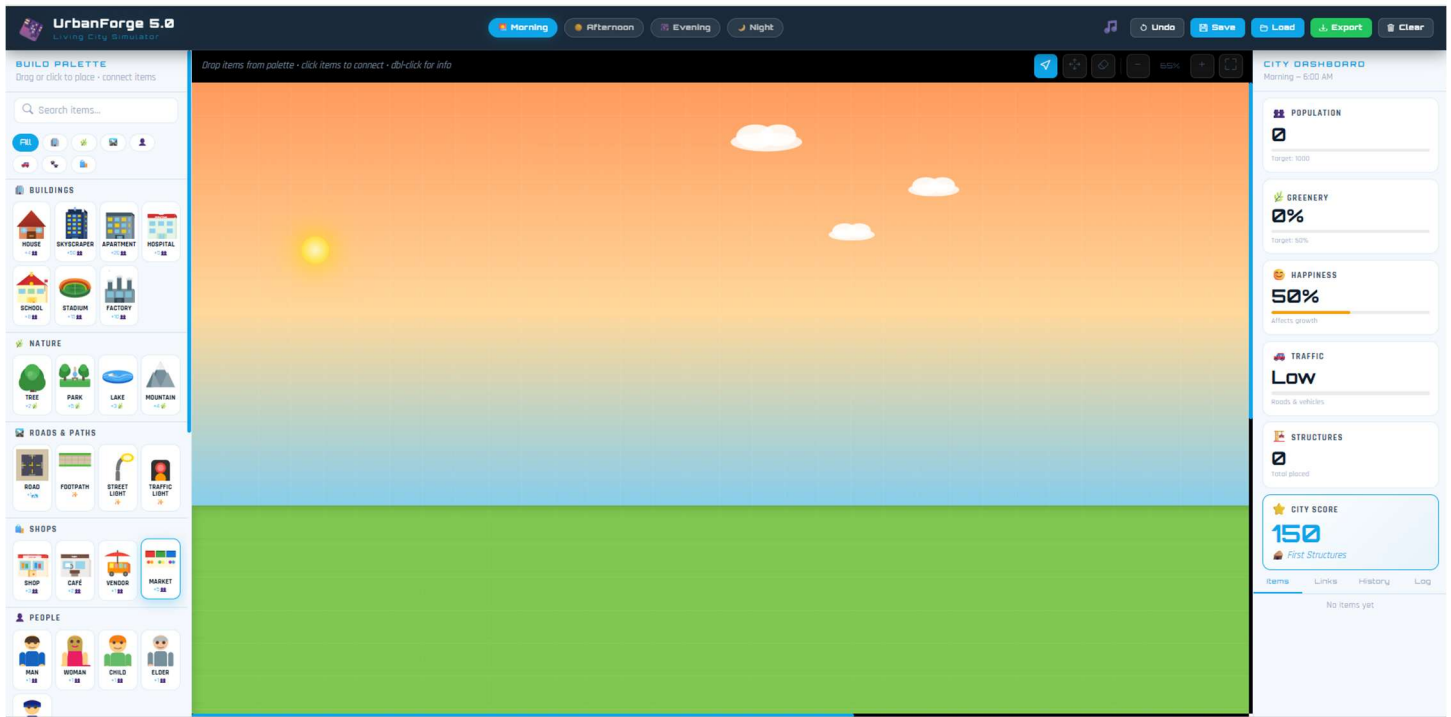


Figure 1 Dashboard

### Dashboard / Stats Panel

The right-side dashboard displays live city stats — Population, Greenery %, Happiness %, Energy balance, City Score with grade, daily Budget with income and upkeep breakdown, and a live city news ticker. Eight tabs (Items, Links, History, Log, Analytics, Research, Policies, Achievements) are all populated with real-time data.

### Canvas — Morning with Road Connections

The 1400×900px canvas shows placed SVG buildings connected by animated road lines with yellow center-line dashes and a traveling dot. The morning sky gradient (amber→light blue→green ground) renders above the grid overlay. The minimap in the bottom-right corner reflects all items and connections.

## Night Mode — Building Glow Effect

In Night time-of-day, the canvas transitions to a near-black sky with 230 twinkling stars rendered on an HTML5 canvas element. All buildings show CSS drop-shadow glow animation simulating lit windows. The minimap and all UI panels update to the dark color scheme via CSS variables.

## History Tab with Thumbnails

After clicking Save, a snapshot card appears in the History tab showing a canvas-generated thumbnail image (buildings as colored dots on the sky gradient background), item/link counts, city score, budget, and timestamp. The Restore button replays the full city state.

## Analytics Tab — Live Charts

The Analytics tab renders live metric rows with animated fill bars for all city statistics — Population (target 1000), Greenery (target 60%), Happiness, Energy, Connections, Tech Unlocked, and Achievement Points — plus a category breakdown of all placed items.

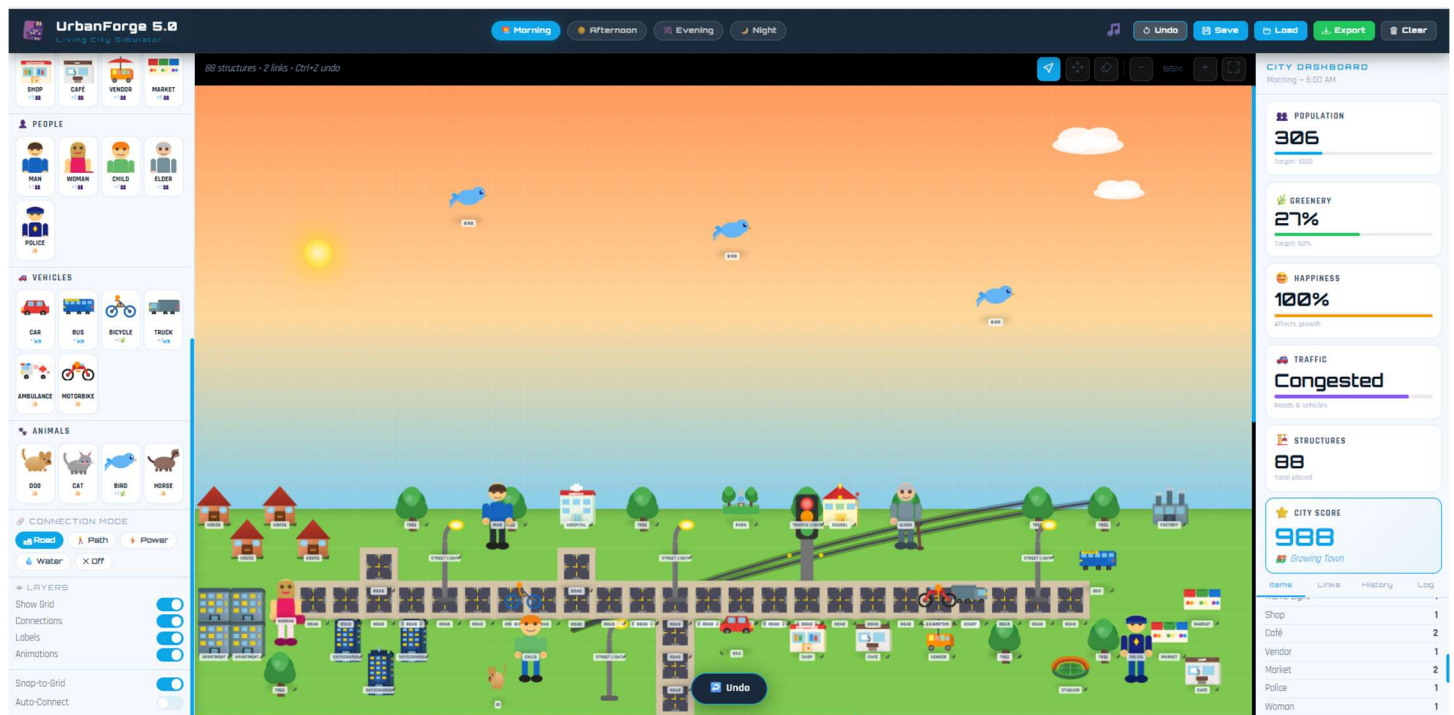


Figure 2 Output

## Conclusion

The UrbanForge 5.0: Advanced Living City AI Simulator successfully demonstrates the practical application of advanced front-end web technologies to create a rich, self-contained interactive simulation. Built entirely in a single HTML file using HTML5, CSS3, JavaScript, jQuery, jQuery UI, and Bootstrap, the application runs directly in any modern browser without any server-side infrastructure or installation.

The system implements a comprehensive economy engine with real budget mechanics, a technology research tree with 8 permanent upgrades, 7 city policies with meaningful tradeoffs, and a disaster simulation system covering storms, fires, earthquakes, and floods — each with unique visual effects using CSS animations, SVG.

Overall, UrbanForge 5.0 provides an excellent demonstration of how complex, game-like simulation systems can be built entirely using client-side web technologies, offering both educational value in front-end engineering and an engaging interactive experience.

## Future Work

The UrbanForge 5.0 simulator can be extended with the following features:

- 1. WebGL / Three.js 3D Rendering** — Transition from 2D flat canvas to a true isometric or full 3D perspective using Three.js, enabling real 3D building models with shadows and camera rotation.
- 2. Multiplayer Mode** — Implement WebSocket-based real-time collaboration where multiple users can build on the same canvas simultaneously.
- 3. AI City Advisor** — Integrate a language model API to provide contextual city planning advice based on current statistics, suggesting optimal building placements and policy

combinations.

**4. Procedural City Generation** — Implement an algorithm that auto-generates realistic city layouts based on selected style templates (Downtown, Suburb, Industrial).

**5. Traffic Flow Simulation** — Model actual vehicle movement along connected roads with congestion, traffic light timing optimization, and public transit ridership.

**6. Water and Power Network Visualizer** — Show live flow of water and electricity through the connection network with capacity indicators and shortage alerts.

## References

- **GitHub Link.** <https://github.com/Nancy5848/UrbanForge-5.0-Advanced-Living-City-AI-Simulator>
- **MDN Web Docs.** (2024). *HTML, CSS, JavaScript, Canvas & Web Audio API.* <https://developer.mozilla.org>
- **jQuery UI.** (2023). *Draggable, Droppable & Resizable Documentation.* <https://jqueryui.com>
- **Bootstrap.** (2024). *Bootstrap 5.3 Framework & Icons.* <https://getbootstrap.com>
- **W3C.** (2021). *SVG 2.0 & Web Storage (localStorage) Specifications.* <https://www.w3.org>
- **Adams, E.** (2014). *Fundamentals of Game Design* (3rd ed.). New Riders Press.
- **GitHub & OpenAI.** (2024). *Copilot, ChatGPT & Gemini AI Tools.* <https://github.com> & <https://openai.com>